Lab 00 — Hands-on lab: a tour of Linux from the terminal

Goal: manipulate every notion from the theory — processes, signals, exit codes, permissions, environment, streams, ports, archives — using nothing but what Ubuntu 24.04 ships with. No container here: everything you see will come back, as is, in the Docker labs.

Prerequisites — Windows 10/11 with WSL 2 and an **Ubuntu 24.04** distribution. Nothing else: no Podman (lab 01), no extra package. Open an Ubuntu terminal and stay in it.

Step 0 — Where am I, and who am I?

```
head -n 2 /etc/os-release
uname -r
whoami
id
```

Observe `PRETTY_NAME="Ubuntu 24.04.x LTS"`, a kernel `6.6.87.2-microsoft-standard-WSL2` (the suffix is the WSL signature), your username, and a line `uid=1000(...) gid=1000(...) groups=...27(sudo) ...`.

Explanation. Three identities never to confuse again: the **distribution** (Ubuntu 24.04, the userland), the **kernel** (compiled by Microsoft for WSL), and **you** (UID 1000, member of the `sudo` group). The kernel will only ever know you as that number, 1000.

> → WINDOWS / WSL
> If `uname -r` does not show `-microsoft-standard-WSL2`, you are not in WSL 2 (`wsl --version` and `wsl --list --verbose` on the PowerShell side to check). The whole lab series assumes WSL 2.

Step 1 — The kernel, the userland, and the frontier

```
cat /proc/version
type ls
type cd
which cat
```

Observe: the kernel version spelled out; `ls is /usr/bin/ls` (a program, a file on disk); `cd is a shell builtin` (not a program: an internal function of the shell); `/usr/bin/cat`.

Explanation. Everything you type is either a **program** from the userland (an executable file somewhere) or a shell builtin. Neither touches the hardware: both go through the kernel's system calls. `cd` is a builtin for a precise reason: changing directory is an attribute *of the shell process itself* — an external program would change its own process's directory, then die, with no effect on you.

> → LINUX
> `type` asks the shell ("what would you do with this word?"); `which` only searches the `PATH`. When in doubt about a command that "lies" (alias, function), `type` always tells the truth.

Step 2 — One tree, several mounts

```
ls /
findmnt /
df -h /
ls /mnt/c/Windows 2>/dev/null | head -n 3
```

Observe the single root (`bin boot dev etc home ... proc ... tmp usr var`), the `findmnt` line of the form `/ /dev/sdc ext4 rw,relatime,...`, a `df` around `1007G` (the *virtual* size of the WSL disk), and — this is Windows seen from Linux — the contents of `C:\Windows`.

```
findmnt -t proc
ls /proc | head -n 8
grep MemTotal /proc/meminfo
```

Observe `/proc proc proc rw,relatime`: a filesystem of type `proc`, with no disk behind it. The `ls` lists **numbers** — one per living process — and `MemTotal` comes straight from the kernel.

Explanation. No `C:`/`D:` drives: everything hangs on the same tree through **mounts**. The Linux disk provides `/`, the Windows disk is mounted on `/mnt/c`, and the kernel itself is mounted on `/proc` — a directory whose files are fabricated on the fly at every read. Docker images (lab 02) and volumes (lab 06) will do nothing but add mounts to this tree.

> **→ WINDOWS / WSL**
> `/mnt/c` crosses a Windows ↔ Linux boundary: it is **slow**. A project you compile, or images you store, must live on the Linux side (`/home/...`), not in `/mnt/c/Users/...`. A reflex to acquire right now.

Step 3 — Processes: PID, parent, /proc

```
ps
echo $$
ps -p 1 -o pid,comm
```

Observe: an almost empty `ps` (your `bash`, the `ps` itself); your shell's PID (`echo $$`); and process 1: `systemd`.

Now launch a process that lasts, in the background:

```
sleep 300 &
ps -o pid,ppid,stat,cmd
```

Observe a `sleep 300` line whose **PPID is your bash's PID**: you have just witnessed a parent-child link.

```
  PID  PPID STAT CMD
 2363  2362 S    bash
 2419  2363 S    sleep 300
 2420  2363 R    ps -o pid,ppid,stat,cmd
```

Go look at this process in `/proc` (replace `2419` with your PID):

```
head -n 3 /proc/2419/status
tr '\0' ' ' < /proc/2419/cmdline; echo
ls -l /proc/2419/exe
```

Observe `Name: sleep`, `State: S (sleeping)`, the exact command line, and a link `exe -> /usr/bin/sleep`.

Explanation. `ps` has nothing magical: it reads `/proc`. Everything Docker will show you later (`podman top`, `podman inspect`) comes from there too. `STAT S` means *sleeping* — waiting; `R`, *running*.

Step 4 — Signals and exit codes

The `sleep` is still running. Dismiss it politely:

```
kill 2419        # your own PID
ps -o pid,cmd | grep "[s]leep 300" || echo "no more sleep process"
```

Observe `Terminated` (printed by the shell) then `no more sleep process`: `kill` with no option sends `SIGTERM`, and `sleep` obeys.

Again, brutally this time:

```
sleep 300 &
kill -9 %1
```

Observe `Killed` this time: `SIGKILL` asked for nothing. (`%1` designates the shell's *job* no. 1 — convenient to avoid hunting for the PID.)

Now, collecting exit codes:

```
true;  echo $?
false; echo $?
ls /does-not-exist; echo $?
unknown-command; echo $?
bash -c 'kill -9 $$'; echo $?
```

Observe, in order: `0`, `1`, `2` (after the error message from `ls`), `127` (after `command not found`), and **`137`** (after `Killed`).

Explanation. `0` = success, the rest = failure, and `128 + n` = death by signal *n*: 137 = 128 + 9 = killed by SIGKILL. These five numbers are exactly what `podman ps` will display in its `Exited (...)` column in lab 03 — learn to read them here, where everything is simple.

> **REMEMBER**
> The civilized escalation: `kill` (SIGTERM, the application may tidy up), wait, then only `kill -9` (SIGKILL, the kernel erases). `docker stop` applies this protocol automatically: SIGTERM, a 10-second grace period, SIGKILL.

Step 5 — The environment and the PATH

```
echo $HOME
env | wc -l
env | grep -E '^(HOME|PATH|LANG)='
```

Observe your environment: a few dozen variables, including `HOME=/home/<you>` and a `PATH` which, on WSL, also contains Windows paths (`/mnt/c/Windows/system32`...).

The decisive experiment — a shell variable is **not** an environment variable:

```
MSG=hello
echo $MSG
bash -c 'echo child sees: [$MSG]'
export MSG
bash -c 'echo child sees: [$MSG]'
```

Observe: `hello`, then `child sees: []` (empty!), then, after `export`, `child sees: [hello]`.

Explanation. Each child process receives a **copy** of its parent's environment, frozen at launch. Before `export`, `MSG` existed only in your shell. This exact mechanism is what `podman run -e MSG=hello` will use in lab 08 to configure your applications.

Next, the `PATH`:

```
mkdir -p ~/lab0/tools
printf '#!/bin/bash\necho "homemade tool: ok"\n' > ~/lab0/tools/mytool
chmod +x ~/lab0/tools/mytool
mytool; echo $?
export PATH="$HOME/lab0/tools:$PATH"
mytool
```

Observe first `command not found` and `127`, then, once the directory is added to the `PATH`, `homemade tool: ok`.

Explanation. The shell "knows" no command: it searches for an executable of that name in the `PATH` directories, in order, and stops at the first one found. (This modified `PATH` only lasts for this shell; permanent = one line in `~/.bashrc`.)

Step 6 — Three streams: redirections and pipes

```
cd ~/lab0
echo "first line"  > notes.txt
echo "second line" >> notes.txt
cat notes.txt
```

Observe: `>` creates (or overwrites!), `>>` appends.

Now separate the two output streams:

```
ls notes.txt /does-not-exist > output.txt 2> errors.txt
cat output.txt
cat errors.txt
```

Observe: the screen stayed silent during the `ls`; `output.txt` contains `notes.txt`, `errors.txt` contains `ls: cannot access '/does-not-exist': No such file or directory`.

```
ls notes.txt /does-not-exist > all.txt 2>&1
cat all.txt
```

Observe the two lines together: `2>&1` plugs the error stream (2) into wherever the output (1) is pointing.

Finally, pipes:

```
ps -ef | wc -l
ps -ef | grep "[b]ash" | head -n 3
```

Observe the system's process count, then your shells — with no intermediate file: each command's output feeds the next one's input.

> **→ LINUX / SHELL**
> The `grep "[b]ash"` trick: the brackets form a regular expression that matches `bash` ... but the `grep`'s own command line contains `[b]ash`, which does not match itself. Without it, `grep` would always find itself. You will see this pattern in every lab.

Step 7 — Permissions: reading an `ls -l`

```
ls -l notes.txt
stat -c "%U %G %a %n" notes.txt
```

Observe `-rw-r--r-- 1 <you> <you> 23 ... notes.txt` and its numeric form `644`: owner `rw` (6), group `r` (4), others `r` (4).

Create a script and try to run it:

```
printf '#!/bin/bash\necho "Hello, I am process $$"\n' > hello.sh
./hello.sh; echo $?
chmod +x hello.sh
ls -l hello.sh
./hello.sh
```

Observe: `Permission denied` and code **126** (found but not executable); then, after `chmod +x`, `-rwxr-xr-x` and the script running — with a different PID at each launch.

And the root frontier:

```
cat /etc/shadow; echo $?
ls -l /etc/shadow
sudo head -n 1 /etc/shadow
```

Observe `Permission denied` (code 1), the line `-rw-r----- 1 root shadow ...` which explains it (you are neither `root` nor in the `shadow` group), then, via `sudo`, the first line `root:*:...` (`*` or `!`: locked account, no password accepted).

Explanation. The kernel compares the process's UID to the three `rwx` triplets and applies the first one that concerns you. `sudo` "bypasses" nothing: it launches the process with UID 0, to which the kernel refuses nothing. In lab 06, when a container writes files into a volume that belong to an unexpected UID, this is the reading grid you will need.

Step 8 — A server, a port, a client

Ubuntu 24.04 ships with Python: your first HTTP server in one line.

```
echo "<h1>Hello from my server</h1>" > index.html
python3 -m http.server 8080 &
curl -s http://localhost:8080/index.html
```

Observe your HTML returned over HTTP: `<h1>Hello from my server</h1>`.

```
curl -si http://localhost:8080/index.html | head -n 4
ss -tlnp | grep 8080
```

Observe the full HTTP response (`HTTP/1.0 200 OK`, `Server: SimpleHTTP/0.6 Python/3.12.3`, `Content-type: text/html`) and the listening line:

```
LISTEN 0  5  0.0.0.0:8080  0.0.0.0:*  users:(("python3",pid=2788,fd=3))
```

`0.0.0.0:8080`: the `python3` process listens on **all** interfaces, port 8080.

> **→ WINDOWS / WSL**
> Open a **Windows** browser at `http://localhost:8080`: the page appears. WSL 2 automatically relays `localhost` from Windows to Ubuntu. This relay is what will let you, in lab 07, test your containers from a Windows browser.

Now try a privileged port:

```
python3 -m http.server 80
```

Observe the failure: `PermissionError: [Errno 13] Permission denied`. Port 80 is below the 1024 threshold, reserved for root — and you are UID 1000. Rootless Podman will inherit the same limit.

Shut the server down and verify:

```
kill %1
curl -s --max-time 2 http://localhost:8080/; echo $?
```

Observe `curl`'s code **7**: *connection refused* — nobody is listening anymore.

Step 9 — Archiving: `tar`, the ancestor of images

```
mkdir -p my-app/config
echo "app.port=8080" > my-app/config/app.properties
echo "fake binary" > my-app/app.bin
tar -czf my-app.tar.gz my-app
ls -lh my-app.tar.gz
file my-app.tar.gz
```

Observe an archive of a few hundred bytes, identified as `gzip compressed data`.

```
tar -tf my-app.tar.gz
mkdir -p /tmp/restore
tar -xzf my-app.tar.gz -C /tmp/restore
cat /tmp/restore/my-app/config/app.properties
```

Observe the content listing (`-t` = *test/list*), then the extraction elsewhere (`-C`) and the file restored identically: `app.port=8080`.

Explanation. `tar` (*tape archive*, 1979) puts a whole tree — paths, permissions, owners — into a single file. Remember it well: a Docker image **layer** is literally a tar archive, and `podman save` (lab 02) will hand you a tar of tars. Nothing new under the sun.

Cleanup

Check that no lab process is lingering, then delete the files:

```
ps -o pid,cmd | grep -E "[s]leep|[h]ttp.server" || echo "nothing to kill"
rm -r ~/lab0
rm -r /tmp/restore
```

The modified `PATH` and the `MSG` variable will vanish with this shell: close the terminal. (Nothing was installed: there is nothing to uninstall.)

What you must be able to state now

- My kernel is `...-microsoft-standard-WSL2`; my distribution is Ubuntu 24.04; I am UID 1000.
- A process has a PID and a parent; I saw it born (`&`), alive (`/proc/<pid>/`), and dead (`kill`).
- `kill` sends SIGTERM (negotiable), `kill -9` SIGKILL (non-negotiable); a process killed by SIGKILL exits with `137` = 128 + 9.
- `$?` is `0` on success; `126` = not executable, `127` = not found in the `PATH`.
- A variable only reaches child processes after `export` — and never processes already running.
- `>` captures stdout, `2>` stderr, `2>&1` merges them, `|` chains processes.
- `-rw-r-----` reads as three triplets; the kernel compares UIDs, and `root` (UID 0) ignores the grid.
- `ss -tlnp` tells me who listens on which port; `0.0.0.0` = all interfaces; < 1024 = root only; `curl` tests the whole thing.

- A mount hooks a filesystem onto the single tree; `/proc` has no disk; `tar` packs up a tree — Docker images will do the same.